Rapport de Cloud DevOps Projet

Déploiement d'une API REST sur Azure avec CI/CD

Auteurs :

LEGNIOUI SIFEDDINE

LOUAT OUSSAMA

AYOUB AFRAOUI

KAWTAR ELBEJJAJI

Encadrant Académique

Mme Faouzia Benabbou

Module: MEIA - Cloud DevOps

25 mars 2026

Résumé

Ce rapport présente un projet complet d'ingénierie DevOps réalisé dans le cadre du module MEIA Cloud DevOps. Le projet "Task Manager API" est une démonstration pratique de l'ensemble de la chaîne DevOps moderne, incluant le développement d'une application, la conteneurisation, l'infrastructure as code, la configuration automatisée et l'orchestration de conteneurs.

L'objectif principal est de déployer une application Flask REST sur une infrastructure cloud (Azure) en utilisant les meilleures pratiques de l'industrie DevOps. Le projet intègre Docker pour la conteneurisation, Terraform pour la provision d'infrastructure, Ansible pour la configuration des serveurs, Kubernetes (K3s) pour l'orchestration des conteneurs, et GitHub Actions pour l'intégration et le déploiement continus.

Ce document fournit une documentation détaillée de tous les composants, des principaux livrables, de la stratégie de déploiement et des scripts utilisés. Il constitue un guide complet pour reproduire l'infrastructure et comprendre les choix architecturaux.

Mots-clés : DevOps, Cloud Computing, Azure, Docker, Kubernetes, Terraform, Ansible, CI/CD, Infrastructure as Code, Orchestration

Table des matières

1 Introduction **9**

1.1 Contexte du projet 9

1.2 Objectifs du projet 9

1.3 Public cible et utilité du document 10

1.4 Structure du rapport 10

2 Architecture Générale du Projet **11**

2.1 Vue d'ensemble de l'architecture 11

2.2 Composants principaux 11

2.2.1 Application Flask 11

2.2.2 Conteneurisation Docker 12

2.2.3 Infrastructure Azure 12

2.2.4 Terraform 12

2.2.5 Ansible 12

2.2.6 Kubernetes (K3s) 12

2.2.7 Pipeline CI/CD 12

2.3 Flux de déploiement 13

2.4 Avantages de cette architecture 13

3 Application Flask - Task Manager API **14**

3.1 Présentation générale 14

3.2 Architecture de l'application 14

3.3 Endpoints API 15

3.4 Structure des données 15

3.5 Code source principal 15

3.5.1 Initialisation et routes principales 15

3.5.2 Opération GET - Récupérer toutes les tâches 16

3.5.3 Opération POST - Créer une tâche 16
3.5.4 Opération PATCH/PUT - Mettre à jour une tâche 16
3.5.5 Opération DELETE - Supprimer une tâche 17
3.6 Interface utilisateur 17
3.7 Dépendances Python 17
3.8 Démarrage de l'application 17

4 Containerisation Docker 19
4.1 Vue d'ensemble de Docker 19
4.2 Dockerfile 19
4.2.1 Analyse du Dockerfile 19
4.2.2 Explications détaillées 20
4.3 Meilleure pratiques appliquées 20
4.4 Construction de l'image 21
4.4.1 Commande de base 21
4.4.2 Construction avec tag complet 21
4.4.3 Avec tag de version 21
4.5 Exécution du conteneur 21
4.5.1 Environnement de développement 21
4.5.2 Environnement de production 21
4.6 Optimisation des images Docker 22
4.6.1 Considérations de taille 22
4.6.2 Multi-stage builds (recommandé) 22
4.7 Registre d'images 22

5 Infrastructure as Code avec Terraform 24
5.1 Introduction à Terraform 24
5.2 Configuration des providers 24
5.2.1 providers.tf 24
5.2.2 Explication des providers 25
5.3 Variables Terraform 25
5.3.1 variables.tf 25
5.4 Ressources principales 26

5.4.1 Resource Group 26
5.4.2 Virtual Network 26
5.4.3 Network Security Group 27
5.5 Outputs 28
5.5.1 outputs.tf 28
5.6 Workflow de déploiement 29
5.6.1 Initialisation 29
5.6.2 Plan 30
5.6.3 Apply 30
5.6.4 Destruction 30
5.7 État Terraform 30
5.7.1 Fichiers d'état 30
5.7.2 Bonnes pratiques 30

6 Configuration Automatisée avec Ansible 31
6.1 Introduction à Ansible 31
6.2 Playbook principal 31
6.2.1 playbook.yml 31
6.2.2 Structure et explication 33
6.3 Fichier d'inventaire 33
6.3.1 inventory.ini.tpl 33
6.4 Exécution du playbook 34
6.4.1 Prérequis 34
6.4.2 Commandes 34
6.5 Variables et fact gathering 34
6.5.1 Utilisation de variables 34
6.5.2 Enregistrement de variables 34
6.6 Bonnes pratiques 35

7 Orchestration Kubernetes (K3s) 36
7.1 Vue d'ensemble de Kubernetes 36
7.1.1 Concepts clés 36
7.2 Manifests Kubernetes 36

7.2.1 Deployment 36
7.2.2 Service 38
7.3 Déploiement sur Kubernetes 39
7.3.1 Prérequis 39
7.3.2 Commandes de déploiement 39
7.3.3 Accès à l'application 40
7.4 Gestion du cycle de vie des pods 40
7.4.1 Mise à jour d'image 40
7.4.2 Rolling deployment 41
7.4.3 Scaling 41
7.5 Health checks et probes 41
7.5.1 Readiness Probe 41
7.5.2 Liveness Probe (recommandé) 41

8 Pipeline CI/CD avec GitHub Actions 43
8.1 Vue d'ensemble 43
8.2 Workflows GitHub Actions 43
8.2.1 Structure générale 43
8.2.2 Exemple de workflow CI/CD 43
8.3 Déclencheurs (triggers) 46
8.3.1 Push sur la branche main 46
8.3.2 Pull requests 46
8.3.3 Schedules 46
8.4 Jobs et dépendances 46
8.4.1 Définition de jobs 46
8.4.2 Conditions 47
8.5 Secrets et variables 47
8.5.1 Déclaration des secrets 47
8.5.2 Utilisation dans le workflow 47
8.6 Actions réutilisables 47
8.6.1 Actions officielles 47
8.6.2 Exemple : actions/checkout 48
8.7 Failure handling 48

8.7.1 Continue on error 48
8.7.2 Conditions avancées 48
8.8 Monitoring et logging 49
8.8.1 Annotations 49
8.8.2 Groupes de logs 49

9 Guide Complet de Déploiement 50
9.1 Prérequis préalables 50
9.1.1 Logiciels à installer 50
9.1.2 Comptes requis 50
9.2 Étape 1 : Préparation de l'environnement 50
9.2.1 1.1 Authentification Azure 50
9.2.2 1.2 Clonage du repository 51
9.2.3 1.3 Configuration Docker 51
9.3 Étape 2 : Déploiement de l'infrastructure avec Terraform 51
9.3.1 2.1 Initialisation Terraform 51
9.3.2 2.2 Planification 51
9.3.3 2.3 Application 52
9.3.4 2.4 Récupération des IPs 52
9.4 Étape 3 : Configuration avec Ansible 52
9.4.1 3.1 Permissions SSH 52
9.4.2 3.2 Désactivation de la vérification SSH 52
9.4.3 3.3 Exécution du playbook 52
9.5 Étape 4 : Configuration de kubectl 52
9.5.1 4.1 Copie du kubeconfig 52
9.5.2 4.2 Verification de la connexion 53
9.5.3 4.3 Vérification des nodes 53
9.6 Étape 5 : Construction de l'image Docker 53
9.6.1 5.1 Build local 53
9.6.2 5.2 Tagging 53
9.6.3 5.3 Push vers Docker Hub 54
9.7 Étape 6 : Déploiement sur Kubernetes 54
9.7.1 6.1 Application des manifests 54

9.7.2 6.2 Vérification du déploiement 54
9.7.3 6.3 Vérification du service 54
9.8 Étape 7 : Accès à l'application 54
9.8.1 7.1 Déterminer l'URL 54
9.8.2 7.2 Test de l'API 55
9.9 Étape 8 : Configuration du pipeline CI/CD 55
9.9.1 8.1 Secrets GitHub 55
9.9.2 8.2 Activation du workflow 55
9.9.3 8.3 Test du workflow 55
9.10 Troubleshooting 56
9.10.1 Erreurs de connexion Terraform 56
9.10.2 Erreurs SSH Ansible 56
9.10.3 Pods non Ready 56

10 Maintenance, Monitoring et Meilleures Pratiques 57
10.1 Monitoring des applications 57
10.1.1 Endpoins de santé 57
10.1.2 Kubernetes health checks 57
10.2 Logging et debugging 58
10.2.1 Logs Kubernetes 58
10.2.2 Inspection des ressources 58
10.3 Gestion des versions 59
10.3.1 Versioning d'images 59
10.3.2 Déploiement de nouvelles versions 59
10.4 Meilleures pratiques 59
10.4.1 Infrastructure 59
10.4.2 Application 60
10.4.3 DevOps 60
10.4.4 Sécurité 60
10.5 Cleanup et destruction 60
10.5.1 Destruction de l'infrastructure 60

11 Conclusions et Perspectives 62

11.1 Résumé du projet 62
11.2 Technologies et outils maîtrisés 62
11.3 Améliorations futures 63
11.3.1 Court terme 63
11.3.2 Moyen terme 63
11.3.3 Long terme 63
11.4 Leçons apprises 63
11.4.1 Points clés 63
11.4.2 Défis rencontrés 64
11.5 Recommandations 64

A Code Source Complet **65**
A.1 Application Flask (app.py) 65

B Configuration Terraform **67**
B.1 Fichier main.tf complet 67

C Manifests Kubernetes **68**
C.1 Déploiement complet 68

D Résumé des commandes **69**
D.1 Commandes essentielles 69

E Ressources et références **70**
E.1 Documentation officielle 70
E.2 Livres recommandés 70
E.3 Community et forums 70

Conclusion Générale **71**

Chapitre 1

Introduction

1.1 Contexte du projet

Le projet "Task Manager API" est conçu comme une application d'apprentissage pratique pour les concepts avancés de DevOps et d'ingénierie cloud. Dans le contexte actuel de digitalisation et de transformations cloud, la maîtrise des technologies et des méthodologies DevOps est devenue essentielle pour tout ingénieur infrastructurePédagogique.

Ce projet couvre l'intégralité de la chaîne de valeur DevOps :

— **Développement** : Une application REST API légère et modulaire

— **Containerisation** : Empaquetage de l'application dans des conteneurs Docker

— **Infrastructure** : Provisioning automatisé d'infrastructure sur Azure via Terraform

— **Configuration** : Automation de configuration serveur via Ansible

— **Orchestration** : Déploiement et gestion des conteneurs via Kubernetes (K3s)

— **CI/CD** : Pipeline automatisé via GitHub Actions

1.2 Objectifs du projet

Les objectifs principaux de ce projet sont :

1. Développer une application REST complète avec Flask
2. Containeriser l'application de manière optimale avec Docker
3. Automatiser le provisioning d'infrastructure cloud (Azure) avec Terraform
4. Configurer automatiquement les serveurs avec Ansible

5. Orchestrer les conteneurs avec Kubernetes (K3s)
6. Implémenter un pipeline CI/CD complet avec GitHub Actions
7. Documenter l'ensemble de l'infrastructure et du processus de déploiement
8. Démontrer les meilleures pratiques DevOps dans un contexte réel

1.3 Public cible et utilité du document

Ce rapport s'adresse à :

— Les étudiants en ingénierie cloud et DevOps

— Les administrateurs systèmes intéressés par l'automatisation

— Les ingénieurs DevOps souhaitant améliorer leurs compétences

— Toute personne intéressée par les architectures cloud modernes

Le document sert de guide complet pour comprendre, reproduire et améliorer la solution proposée.

1.4 Structure du rapport

Chapitre 2 Fournit une vue globale de l'architecture et des composants

Chapitre 3 Détaille l'application Flask développée

Chapitre 4 Explique la containerisation Docker

Chapitre 5 Décrit l'infrastructure as code avec Terraform

Chapitre 6 Explique la configuration automatisée avec Ansible

Chapitre 7 Détaille l'orchestration Kubernetes

Chapitre 8 Présente le pipeline CI/CD

Chapitre 9 Fournit des instructions de déploiement

Chapitre 10 Couvre les aspects de maintenance et monitoring

Annexes Code source complet et configurations

Chapitre 2

Architecture Générale du Projet

2.1 Vue d'ensemble de l'architecture

Le projet suit une architecture moderne et modulaire exploitant les principes de l'Infrastructure as Code, de la Conteneurisation et de l'Orchestration. La figure ci-dessous illustre les différentes couches de l'architecture.

2.2 Composants principaux

2.2.1 Application Flask

Une application REST légère écrite en Python utilisant le framework Flask. Elle expose des endpoints pour la gestion de tâches (CRUD operations).

2.2.2 Conteneurisation Docker

Docker est utilisé pour packager l'application avec toutes ses dépendances dans un conteneur isolé et reproductible. Cela assure la cohérence entre les environnements de développement, test et production.

2.2.3 Infrastructure Azure

L'infrastructure est hébergée sur Microsoft Azure, utilisant :

— Resource Groups pour l'organisation logique

— Virtual Networks (VNet) pour la communication sécurisée

— Network Security Groups (NSG) pour le contrôle d'accès

— Virtual Machines (VMs) Linux Ubuntu pour les nœuds

2.2.4 Terraform

Infrastructure as Code (IaC) pour l'automatisation complète du provisioning Azure. Permet une réproductibilité et une gestion de version de l'infrastructure.

2.2.5 Ansible

Automation de configuration pour :

— Installation de Docker sur chaque nœud

— Installation et configuration de K3s (Kubernetes léger)

— Jointure des workers au cluster master

2.2.6 Kubernetes (K3s)

Orchestration de conteneurs utilisant K3s, une distribution Kubernetes légère idéale pour les environnements edge et de test.

2.2.7 Pipeline CI/CD

GitHub Actions automatise :

— Les tests et linting du code

— La construction de l'image Docker

— Le push vers Docker Hub
— Le déploiement sur le cluster Kubernetes

2.3 Flux de déploiement

1. Les développeurs poussent le code vers GitHub
2. GitHub Actions déclenche automatiquement le pipeline CI/CD
3. L'application est testée et construite
4. L'image Docker est créée et poussée vers Docker Hub
5. Kubernetes récupère la nouvelle image
6. Les pods sont mis à jour avec le rolling deployment
7. L'application est accessible via le service NodePort

2.4 Avantages de cette architecture

— **Scalabilité** : Kubernetes permet de scaler l'application facilement
— **Automatisation** : Terraform et Ansible automatisent tout le provisioning
— **Reproductibilité** : Infrastructure as Code garantit la reproductibilité
— **Continuous Delivery** : Pipeline CI/CD pour des déploiements rapides
— **Résilience** : K3s assure la haute disponibilité
— **Isolation** : Docker isolate complètement l'application
— **Flexibilité** : Facile à adapter et à modifier

Chapitre 3

Application Flask - Task Manager API

3.1 Présentation générale

L'application Task Manager API est une API REST complète écrite en Python avec le framework Flask. Elle permet de gérer une liste de tâches avec les opérations CRUD (Create, Read, Update, Delete) standard.

3.2 Architecture de l'application

L'application suit une architecture simple et monolithique appropriée pour une démonstration DevOps. Elle utilise :

— **Flask** : Framework web minimaliste et flexible
— **Python 3.9** : Langage de programmation moderne
— **Gunicorn** : Serveur WSGI pour la production
— **UUID** : Pour la génération d'identifiants uniques

3.3 Endpoints API

Table 3.1 – Endpoints de l'API Task Manager

Méthode	Endpoint	Description	Code
GET	`/`	Retourne la page HTML	200
GET	`/tasks`	Liste toutes les tâches	200
POST	`/tasks`	Crée une nouvelle tâche	201
PATCH/PUT	`/tasks/<id>`	Met à jour une tâche	200/404
DELETE	`/tasks/<id>`	Supprime une tâche	200/404
GET	`/health`	Health check	200

3.4 Structure des données

Une tâche est représentée par la structure JSON suivante :

```
{
    "id": "uuid-string",
    "title": "Titre de la tche",
    "description": "Description optionnelle",
    "done": false
}
```

3.5 Code source principal

3.5.1 Initialisation et routes principales

```
from flask import Flask, jsonify, request, render_template
import uuid

app = Flask(__name__)
tasks = []

@app.route('/')
def index():
```

```
    return render_template('index.html')

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200
```

3.5.2 Opération GET - Récupérer toutes les tâches

```
@app.route('/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks), 200
```

Cette route retourne toutes les tâches actuellement stockées dans la liste en mémoire.

3.5.3 Opération POST - Créer une tâche

```
@app.route('/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    if not data or not 'title' in data:
        return jsonify({'error': 'Title is required'}), 400

    task = {
        'id': str(uuid.uuid4()),
        'title': data['title'],
        'description': data.get('description', ''),
        'done': False
    }
    tasks.append(task)
    return jsonify(task), 201
```

3.5.4 Opération PATCH/PUT - Mettre à jour une tâche

```
@app.route('/tasks/<task_id>', methods=['PATCH', 'PUT'])
def update_task(task_id):
```

```
    data = request.get_json()
    for task in tasks:
        if task['id'] == task_id:
            if 'done' in data:
                task['done'] = data['done']
            return jsonify(task), 200
    return jsonify({'error': 'Task not found'}), 404
```

3.5.5 Opération DELETE - Supprimer une tâche

```
@app.route('/tasks/<task_id>', methods=['DELETE'])
def delete_task(task_id):
    global tasks
    tasks = [task for task in tasks if task['id'] != task_id]
    return jsonify({'result': True}), 200
```

3.6 Interface utilisateur

L'application inclut une interface web moderne avec :

- Design responsive utilisant CSS moderne
- Upload de tâches via formulaire
- Affichage dynamique via JavaScript
- Icones Font Awesome pour une meilleure UX

3.7 Dépendances Python

```
Flask==3.0.0
Werkzeug==3.0.1
gunicorn==21.2.0
```

3.8 Démarrage de l'application

En développement (avec Flask) :

```
python app.py
```

En production (avec Gunicorn) :

```
gunicorn --bind 0.0.0.0:5000 app:app
```

Chapitre 4

Containerisation Docker

4.1 Vue d'ensemble de Docker

Docker est une plateforme de containerisation qui permet d'empaqueter une application avec toutes ses dépendances dans un conteneur isolé. Les avantages principaux sont :

- **Reproducibilité** : Même comportement sur tous les environnements
- **Isolation** : Application isolée du système hôte
- **Légèreté** : Plus léger qu'une machine virtuelle
- **Portabilité** : Fonctionne sur tout système avec Docker
- **Scalabilité** : Facile de scaler avec l'orchestration

4.2 Dockerfile

Le Dockerfile définit comment construire l'image Docker de notre application.

4.2.1 Analyse du Dockerfile

```
# Base image
FROM python:3.9-slim

# Rpertoire de travail
WORKDIR /app

```

```
# Copie des requirements
COPY app/requirements.txt .

# Installation des dpendances
RUN pip install --no-cache-dir -r requirements.txt

# Copie du code source
COPY app/ .

# Port expos
EXPOSE 5000

# Commande de dmarrage
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

4.2.2 Explications détaillées

FROM Utilise l'image officielle Python 3.9-slim comme base (image minimale pour réduire la taille)

WORKDIR Définit le répertoire de travail à `/app` dans le conteneur

COPY requirements.txt Copie le fichier des dépendances

RUN pip install Installe les dépendances Python (–no-cache-dir réduit la taille)

COPY app/ Copie le code source complet

EXPOSE Déclare le port 5000 (informatif, ne fait pas d'exposition réelle)

CMD Commande à exécuter au démarrage du conteneur

4.3 Meilleure pratiques appliquées

1. **Image de base minimaliste** : Utilisation de python :3.9-slim au lieu de python :3.9
2. **Layer caching** : Dépendances copiées avant le code pour meilleur caching
3. **Réduction de la taille** : –no-cache-dir lors de pip install

4. **Utilisateur non-root** : (Peut être amélioré)
5. **Serveur de production** : Gunicorn plutôt que Flask en développement
6. **Single responsibility** : Un conteneur pour une seule application

4.4 Construction de l'image

4.4.1 Commande de base

```
docker build -t task-manager:latest .
```

4.4.2 Construction avec tag complet

```
docker build -t seiflegnioui/task-manager:latest .
```

4.4.3 Avec tag de version

```
docker build -t seiflegnioui/task-manager:v1.0.0 .
```

4.5 Exécution du conteneur

4.5.1 Environnement de développement

```
docker run -p 5000:5000 task-manager:latest
```

4.5.2 Environnement de production

```
docker run -d \
  --name task-manager \
  -p 80:5000 \
  --restart unless-stopped \
  task-manager:latest
```

4.6 Optimisation des images Docker

4.6.1 Considérations de taille

TABLE 4.1 – Taille estimée de l'image Docker

Composant	**Taille**
Base Python 3.9-slim	$\sim$ 150 MB
Dépendances Python	$\sim$ 50 MB
Code application	$\sim$ 1 MB
Total	$\sim$ 200 MB

4.6.2 Multi-stage builds (recommandé)

Une approche avancée utiliserait un multi-stage build :

```
# Stage 1: Builder
FROM python:3.9-slim as builder
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Runtime
FROM python:3.9-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY app/ .
ENV PATH=/root/.local/bin:$PATH
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

4.7 Registre d'images

Les images sont stockées dans Docker Hub pour faciliter le déploiement :

```
# Login
docker login -u username
```

```

# Push
docker push seiflegnioui/task-manager:latest

# Pull
docker pull seiflegnioui/task-manager:latest
```

Chapitre 5

Infrastructure as Code avec Terraform

5.1 Introduction à Terraform

Terraform est un outil de Infrastructure as Code (IaC) qui permet de décrire et de gérer l'infrastructure cloud en tant que code. Les avantages principaux sont :

— **Déclaratif** : Description de l'état désiré

— **Versionnable** : Infrastructure versionnée comme du code

— **Réutilisable** : Modules réutilisables

— **Prédictif** : Aperçu des changements avant application

— **Multi-cloud** : Support de plusieurs fournisseurs

5.2 Configuration des providers

5.2.1 providers.tf

```
terraform {
  required_version = ">= 1.0.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
    tls = {
```

```
      source = "hashicorp/tls"
      version = "~> 4.0"
    }
    local = {
      source = "hashicorp/local"
      version = "~> 2.1"
    }
  }
}

provider "azurerm" {
  features {}
}
```

5.2.2 Explication des providers

azurerm Provider Microsoft Azure pour la gestion des ressources Azure

tls Provider pour la génération de clés et certificats TLS/SSL

local Provider pour la gestion des fichiers locaux (inventory Ansible)

5.3 Variables Terraform

5.3.1 variables.tf

```
variable "resource_group_name" {
  type = string
  description = "Name of the resource group"
  default = "rg-devops-project"
}

variable "location" {
  type = string
  description = "Azure region for the resources"
  default = "swedencentral"
```

```
}

variable "admin_username" {
  type = string
  description = "Admin username for the VMs"
  default = "azureuser"
}

variable "vm_size_master" {
  type = string
  description = "Size of the Master VM"
  default = "Standard_D2s_v3"
}

variable "vm_size_worker" {
  type = string
  description = "Size of the Worker VM"
  default = "Standard_D2s_v3"
}
```

5.4 Ressources principales

5.4.1 Resource Group

```
resource "azurerm_resource_group" "rg" {
  name = var.resource_group_name
  location = var.location
}
```

5.4.2 Virtual Network

```
resource "azurerm_virtual_network" "vnet" {
  name = "vnet-devops"
  address_space = ["10.0.0.0/16"]
```

```
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name = "subnet-devops"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes = ["10.0.1.0/24"]

  depends_on = [
    azurerm_virtual_network.vnet
  ]
}
```

5.4.3 Network Security Group

```
resource "azurerm_network_security_group" "nsg" {
  name = "nsg-devops"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name = "SSH"
    priority = 1001
    direction = "Inbound"
    access = "Allow"
    protocol = "Tcp"
    source_port_range = "*"
    destination_port_range = "22"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }

```

```
  security_rule {
    name = "KubeAPI"
    priority = 1002
    direction = "Inbound"
    access = "Allow"
    protocol = "Tcp"
    source_port_range = "*"
    destination_port_range = "6443"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name = "HTTP"
    priority = 1003
    direction = "Inbound"
    access = "Allow"
    protocol = "Tcp"
    source_port_range = "*"
    destination_port_range = "30080"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}
```

5.5 Outputs

5.5.1 outputs.tf

```
data "azurerm_public_ip" "master" {
  name = azurerm_public_ip.master_ip.name
  resource_group_name =
      azurerm_linux_virtual_machine.master.resource_group_name
}
```

```
5
6 data "azurerm_public_ip" "worker" {
7   name = azurerm_public_ip.worker_ip.name
8   resource_group_name =
        azurerm_linux_virtual_machine.worker.resource_group_name
9 }
10
11 output "master_public_ip" {
12   value = data.azurerm_public_ip.master.ip_address
13 }
14
15 output "worker_public_ip" {
16   value = data.azurerm_public_ip.worker.ip_address
17 }
18
19 # Generate Ansible Inventory dynamically
20 resource "local_file" "ansible_inventory" {
21   content = templatefile("${path.module}/inventory.ini.tpl", {
22     master_ip = data.azurerm_public_ip.master.ip_address
23     worker_ip = data.azurerm_public_ip.worker.ip_address
24     user = var.admin_username
25   })
26   filename = "${path.module}/../ansible/inventory.ini"
27 }
```

5.6 Workflow de déploiement

5.6.1 Initialisation

```
1 cd terraform
2 terraform init
```

Cette commande :

— Télécharge les providers requis

— Initialise l'état Terraform

— Prépare le répertoire pour les opérations

5.6.2 Plan

```
terraform plan
```

Affiche les changements qui seront appliqués.

5.6.3 Apply

```
terraform apply --auto-approve
```

Crée les ressources Azure.

5.6.4 Destruction

```
terraform destroy --auto-approve
```

Supprime toutes les ressources créées.

5.7 État Terraform

5.7.1 Fichiers d'état

— **terraform.tfstate** : État courant

— **terraform.tfstate.backup** : Sauvegarde de l'état précédent

5.7.2 Bonnes pratiques

— Toujours faire des backups de l'état

— Stocker l'état dans un backend sécurisé (Azure Storage)

— Ne jamais committer l'état dans Git

— Utiliser des locks pour les opérations concurrentes

Chapitre 6

Configuration Automatisée avec Ansible

6.1 Introduction à Ansible

Ansible est un outil de configuration management et automatisation de l'IT. Pour ce projet :

- Installation de Docker sur tous les nœuds
- Installation de K3s sur le nœud master
- Jointure des workers au cluster master
- Récupération du kubeconfig

6.2 Playbook principal

6.2.1 playbook.yml

```
---
- name: Setup K3s Master
  hosts: master
  become: yes
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
```

```yaml
      state: present
      update_cache: yes

  - name: Download and install K3s on Master
    shell: curl -sfL https://get.k3s.io | sh -s - server --docker

  - name: Get K3s token
    slurp:
      src: /var/lib/rancher/k3s/server/node-token
    register: k3s_token

  - name: Fetch Kubeconfig to local machine
    fetch:
      src: /etc/rancher/k3s/k3s.yaml
      dest: ../kubeconfig.yaml
      flat: yes

- name: Setup K3s Worker
  hosts: worker
  become: yes
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
        state: present
        update_cache: yes

    - name: Join K3s Node to Cluster
      shell: "curl -sfL https://get.k3s.io | K3S_URL=https://{{
          hostvars[groups['master'][0]]['inventory_hostname'] }}:6443
          K3S_TOKEN={{ hostvars[groups['master'][0]]['k3s_token']['content']
          | b64decode | trim }} sh -s - --docker"
```

6.2.2 Structure et explication

Le playbook est organisé en deux play :

Play 1 : Setup K3s Master

1. **Install Docker** : Installe Docker via apt
2. **Install K3s** : Télécharge et installe K3s en mode serveur
3. **Get Token** : Récupère le token d'authentification du cluster
4. **Fetch Kubeconfig** : Récupère le fichier kubeconfig pour la machine locale

Play 2 : Setup K3s Worker

1. **Install Docker** : Installe Docker via apt
2. **Join Cluster** : Rejoint le cluster master en utilisant l'URL et le token

6.3 Fichier d'inventaire

6.3.1 inventory.ini.tpl

```
[master]
${master_ip} ansible_user=${user}
    ansible_ssh_private_key_file=../terraform/id_rsa_devops
    ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[worker]
${worker_ip} ansible_user=${user}
    ansible_ssh_private_key_file=../terraform/id_rsa_devops
    ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[k3s_cluster:children]
master
worker
```

6.4 Exécution du playbook

6.4.1 Prérequis

- Ansible installé localement
- SSH configuré avec la clé privée
- Inventory.ini généré par Terraform

6.4.2 Commandes

Première exécution avec vérification SSH

```
cd ansible
chmod 600 ../terraform/id_rsa_devops
export ANSIBLE_HOST_KEY_CHECKING=False
ansible-playbook -i inventory.ini playbook.yml
```

Exécutions ultérieures

```
ansible-playbook -i inventory.ini playbook.yml
```

Mode verbose pour debug

```
ansible-playbook -i inventory.ini playbook.yml -vvv
```

6.5 Variables et fact gathering

6.5.1 Utilisation de variables

- **hostvars** : Accès aux variables de hosts
- **groups** : Accès aux groupes d'inventory
- **b64decode** : Décodage de base64 pour le token

6.5.2 Enregistrement de variables

```
- name: Get K3s token
  slurp:
    src: /var/lib/rancher/k3s/server/node-token
  register: k3s_token
```

La variable `k3s_token` peut être utilisée par les plays suivants.

6.6 Bonnes pratiques

— Utilise `become: yes` pour l'escalade de privilèges

— Diable la vérification SSH pour la première connexion

— Utilise `slurp` pour lire les fichiers distants

— Utilise `fetch` pour télécharger des fichiers

— Organise les tâches de manière logique et lisible

Chapitre 7

Orchestration Kubernetes (K3s)

7.1 Vue d'ensemble de Kubernetes

Kubernetes est une plateforme d'orchestration de conteneurs qui automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. Pour ce projet, nous utilisons K3s (Kubernetes léger).

7.1.1 Concepts clés

Pod Unité de déploiement la plus petite, contenant un ou plusieurs conteneurs

Deployment Contrôleur Kubernetes gérant les réplicas de pods

Service Exposant les pods en tant que service réseau

Node Machine (VM) exécutant les pods

Cluster Ensemble de nodes Kubernetes

kubelet Agent exécutant les pods sur chaque node

API Server Interface de contrôle du cluster

7.2 Manifests Kubernetes

7.2.1 Deployment

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-manager-api
  labels:
    app: task-manager
spec:
  replicas: 2
  selector:
    matchLabels:
      app: task-manager
  template:
    metadata:
      labels:
        app: task-manager
    spec:
      containers:
      - name: task-manager-api
        image: seiflegnioui/task-manager:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 5000
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "250m"
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 10
```

```
35          periodSeconds: 5
```

Explications détaillées

apiVersion Version de l'API Kubernetes utilisée (apps/v1 pour Deployments)

kind Type de ressource Kubernetes (Deployment)

metadata Informations sur le déploiement (name, labels)

spec.replicas Nombre d'instances de l'application (2)

selector Sélecteur pour les pods gérés par ce déploiement

template Template de pod définissant la configuration

containers[].image Image Docker à déployer

imagePullPolicy Always = toujours tirer la dernière image

resources.requests Ressources minimales garanties

resources.limits Ressources maximales autorisées

readinessProbe Sonde de santé pour vérifier que l'app est prête

7.2.2 Service

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: task-manager-service
spec:
  type: NodePort
  selector:
    app: task-manager
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 30080
```

Types de service

TABLE 7.1 – Types de service Kubernetes

Type	Description	Usage
ClusterIP	Service interne au cluster	Communication interne
NodePort	Expose sur tous les nodes	Accès temps de test
LoadBalancer	Load balancer cloud	Production
ExternalName	Alias DNS externe	Services externes

Pour ce projet, nous utilisons **NodePort** qui expose le service sur le port 30080 de chaque node.

7.3 Déploiement sur Kubernetes

7.3.1 Prérequis

- Cluster K3s fonctionnel (installé par Ansible)
- `kubectl` configuré avec kubeconfig.yaml
- Image Docker disponible dans Docker Hub

7.3.2 Commandes de déploiement

Appliquer les manifests

```
# Depuis le rpertoire racine
kubectl apply -f kubernetes/

# Ou individuellement
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
```

Vérifier le déploiement

```
# Lister les deployments
kubectl get deployments
```

```

# Lister les pods
kubectl get pods

# Lister les services
kubectl get services

# Dtails d'un deployment
kubectl describe deployment task-manager-api

# Logs d'un pod
kubectl logs <pod-name>

# Logs en temps rel
kubectl logs -f <pod-name>
```

7.3.3 Accès à l'application

L'application est accessible via :

— **URL** : `http://<master-ip>:30080`

— **API** : `http://<master-ip>:30080/tasks`

7.4 Gestion du cycle de vie des pods

7.4.1 Mise à jour d'image

```
# Mise  jour du dploiement
kubectl set image deployment/task-manager-api \
  task-manager-api=seiflegnioui/task-manager:v2.0.0

# Ou avec kubectl apply aprs modification du YAML
kubectl apply -f kubernetes/deployment.yaml
```

7.4.2 Rolling deployment

Kubernetes effectue un rolling deployment par défaut :

- Les anciens pods sont arrêtés progressivement
- Les nouveaux pods sont créés progressivement
- Zéro downtime garanti

7.4.3 Scaling

```
# Scaler 5 replicas
kubectl scale deployment task-manager-api --replicas=5

# Ou modifier le YAML
kubectl set env deployment/task-manager-api REPLICAS=5
```

7.5 Health checks et probes

7.5.1 Readiness Probe

Utilisé pour déterminer si le pod est prêt à recevoir du trafic :

```
readinessProbe:
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 10
  periodSeconds: 5
```

7.5.2 Liveness Probe (recommandé)

```
livenessProbe:
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 30
```

```
6   periodSeconds: 10
7   failureThreshold: 3
```

La liveness probe redémarre le pod si l'application ne répond pas.

Chapitre 8

Pipeline CI/CD avec GitHub Actions

8.1 Vue d'ensemble

Le pipeline CI/CD automatise les étapes suivantes :

1. Linting et tests du code
2. Construction de l'image Docker
3. Push vers Docker Hub
4. Déploiement sur le cluster Kubernetes

8.2 Workflows GitHub Actions

8.2.1 Structure générale

Une pipeline GitHub Actions est définie dans `.github/workflows/` avec les composants :

name Nom du workflow

on Événements déclencheurs

jobs Travaux à exécuter

steps Étapes individuelles

8.2.2 Exemple de workflow CI/CD

```yaml
name: Build and Deploy

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: 3.9

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r app/requirements.txt
        pip install pytest pylint

    - name: Lint with pylint
      run: |
        pylint app/app.py --disable=C0111,W0612 || true

    - name: Test with pytest
      run: |
        pytest app/ -v || true

```

```yaml
deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'

  steps:
  - uses: actions/checkout@v3

  - name: Set up Docker
    uses: docker/setup-buildx-action@v2

  - name: Login to Docker Hub
    uses: docker/login-action@v2
    with:
      username: \${{ secrets.DOCKER_USERNAME }}
      password: \${{ secrets.DOCKER_PASSWORD }}

  - name: Build and push
    uses: docker/build-push-action@v4
    with:
      context: .
      push: true
      tags: |
        \${{ secrets.DOCKER_USERNAME }}/task-manager:latest
        \${{ secrets.DOCKER_USERNAME }}/task-manager:v\${{
            github.run_number }}

  - name: Deploy to Kubernetes
    run: |
      mkdir -p \$HOME/.kube
      echo "\${{ secrets.KUBECONFIG }}" | base64 -d > \$HOME/.kube/config
      kubectl apply -f kubernetes/
      kubectl rollout restart deployment/task-manager-api
```

8.3 Déclencheurs (triggers)

8.3.1 Push sur la branche main

```
on:
  push:
    branches: [ main ]
```

Le workflow s'exécute automatiquement à chaque push.

8.3.2 Pull requests

```
on:
  pull_request:
    branches: [ main ]
```

Le workflow s'exécute sur chaque PR.

8.3.3 Schedules

```
on:
  schedule:
    - cron: '0 2 * * *' # Quotidien  2h du matin
```

8.4 Jobs et dépendances

8.4.1 Définition de jobs

```
jobs:
  build:
    runs-on: ubuntu-latest
    # ... steps ...

  deploy:
    needs: build # Dpens de build
    runs-on: ubuntu-latest
```

```
    # ... steps ...
```

8.4.2 Conditions

```
deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' # Seulement sur main
```

8.5 Secrets et variables

8.5.1 Déclaration des secrets

Dans GitHub : Settings → Secrets and variables → Actions

Secrets nécessaires :

— `DOCKER_USERNAME` : Username Docker Hub

— `DOCKER_PASSWORD` : Token Docker Hub

— `KUBECONFIG` : Fichier kubeconfig en base64

8.5.2 Utilisation dans le workflow

```
- name: Login to Docker Hub
  uses: docker/login-action@v2
  with:
    username: \${{ secrets.DOCKER_USERNAME }}
    password: \${{ secrets.DOCKER_PASSWORD }}
```

8.6 Actions réutilisables

8.6.1 Actions officielles

— `actions/checkout` : Cloner le repo

— `actions/setup-python` : Installer Python

— `docker/login-action` : Login Docker

— `docker/build-push-action` : Build et push

8.6.2 Exemple : actions/checkout

```
- uses: actions/checkout@v3
  with:
    fetch-depth: 0 # Fetch tous les commits
```

8.7 Failure handling

8.7.1 Continue on error

```
- name: Lint
  run: pylint app/
  continue-on-error: true # Ne pas arrter si erreur
```

8.7.2 Conditions avancées

```
- name: Step conditionnelle
  if: success() # S'il n'y a pas eu d'erreur
  run: echo "Success!"

- name: Autre step conditionnelle
  if: failure() # S'il y a eu une erreur
  run: echo "Failed!"

- name: Always run
  if: always() # Toujours excuter
  run: echo "Done"
```

8.8 Monitoring et logging

8.8.1 Annotations

```
echo "::notice file=app.py,line=10::Found issue"
echo "::warning file=app.py,line=20::Be careful"
echo "::error file=app.py,line=30::Critical error"
```

8.8.2 Groupes de logs

```
echo "::group::Titre du groupe"
# ... commandes ...
echo "::endgroup::"
```

Chapitre 9

Guide Complet de Déploiement

9.1 Prérequis préalables

9.1.1 Logiciels à installer

Table 9.1 – Logiciels requis

Logiciel	Version min	Lien
Azure CLI	2.40+	microsoft.com/cli
Terraform	1.0+	terraform.io
Ansible	2.10+	ansible.com
Docker	20.10+	docker.com
kubectl	1.20+	kubernetes.io
Git	2.30+	git-scm.com

9.1.2 Comptes requis

- Compte Microsoft Azure (Free Tier accepté)
- Compte GitHub
- Compte Docker Hub

9.2 Étape 1 : Préparation de l'environnement

9.2.1 1.1 Authentification Azure

```
# Login
az login

# Configurer la souscription par dfaut
az account set --subscription "YOUR_SUBSCRIPTION_ID"

# Vrifier
az account show
```

9.2.2 1.2 Clonage du repository

```
git clone https://github.com/username/devops-project.git
cd devops-project
```

9.2.3 1.3 Configuration Docker

```
docker login
# Entrez vos credentials Docker Hub
```

9.3 Étape 2 : Déploiement de l'infrastructure avec Terraform

9.3.1 2.1 Initialisation Terraform

```
cd terraform
terraform init
```

Terraform télécharge les providers requis.

9.3.2 2.2 Planification

```
terraform plan
```

Affiche les ressources qui seront créées.

9.3.3 2.3 Application

```
terraform apply --auto-approve
```

Temps estimé : 10-15 minutes.

9.3.4 2.4 Récupération des IPs

```
terraform output
```

Les IPs publiques des VMs seront affichées.

9.4 Étape 3 : Configuration avec Ansible

9.4.1 3.1 Permissions SSH

```
cd ../ansible
chmod 600 ../terraform/id_rsa_devops
```

9.4.2 3.2 Désactivation de la vérification SSH

```
export ANSIBLE_HOST_KEY_CHECKING=False
```

9.4.3 3.3 Exécution du playbook

```
ansible-playbook -i inventory.ini playbook.yml
```

Temps estimé : 10-15 minutes.

9.5 Étape 4 : Configuration de kubectl

9.5.1 4.1 Copie du kubeconfig

Le fichier kubeconfig est automatiquement récupéré par Terraform et Ansible :

```
# Dplacer vers le rpertoire kubectl standard
mkdir -p ~/.kube
cp kubeconfig.yaml ~/.kube/config
```

9.5.2 4.2 Verification de la connexion

```
kubectl cluster-info

# Doit afficher quelque chose comme:
# Kubernetes control-plane is running at https://...
```

9.5.3 4.3 Vérification des nodes

```
kubectl get nodes

# Doit afficher:
# NAME STATUS ROLES READY
# <master-ip> Ready control-plane,master 1/1
# <worker-ip> Ready <none> 1/1
```

9.6 Étape 5 : Construction de l'image Docker

9.6.1 5.1 Build local

```
docker build -t task-manager:latest .
```

9.6.2 5.2 Tagging

```
docker tag task-manager:latest \
  seiflegnioui/task-manager:latest
```

9.6.3 5.3 Push vers Docker Hub

```
docker push seiflegnioui/task-manager:latest
```

9.7 Étape 6 : Déploiement sur Kubernetes

9.7.1 6.1 Application des manifests

```
kubectl apply -f kubernetes/
```

9.7.2 6.2 Vérification du déploiement

```
# Attendre que les pods soient Ready
kubectl get pods -w

# Ctrl+C pour quitter
```

9.7.3 6.3 Vérification du service

```
kubectl get svc

# Doit afficher task-manager-service sur le port 30080
```

9.8 Étape 7 : Accès à l'application

9.8.1 7.1 Déterminer l'URL

```
# Rcuprer l'IP du master
MASTER_IP=\$(terraform output -raw master_public_ip)
echo "Application accessible : http://\$MASTER_IP:30080"
```

9.8.2 7.2 Test de l'API

```
# Rcuprer les tches
curl http://\$MASTER_IP:30080/tasks

# Crer une tche
curl -X POST http://\$MASTER_IP:30080/tasks \
  -H "Content-Type: application/json" \
  -d '{\"title\": \"Test Task\", \"description\": \"Description\"}'

# Vrifier la sant
curl http://\$MASTER_IP:30080/health
```

9.9 Étape 8 : Configuration du pipeline CI/CD

9.9.1 8.1 Secrets GitHub

Sur GitHub, aller à Settings → Secrets and variables → Actions :

1. **DOCKER_USERNAME** : Votre username Docker Hub
2. **DOCKER_PASSWORD** : Votre token Docker Hub
3. **KUBECONFIG** : Contenu base64 de kubeconfig.yaml

Générer la version base64 :

```
cat kubeconfig.yaml | base64 -w 0
```

9.9.2 8.2 Activation du workflow

Créer le fichier `.github/workflows/deploy.yml` avec la configuration du workflow.

9.9.3 8.3 Test du workflow

```
git push origin main
```

Le workflow s'exécute automatiquement.

9.10 Troubleshooting

9.10.1 Erreurs de connexion Terraform

```
# Vrifier l'authentification Azure
az account show

# Rinitialiser Terraform
rm -rf .terraform .terraform.lock.hcl
terraform init
```

9.10.2 Erreurs SSH Ansible

```
# Tester la connectivit SSH
ssh -i terraform/id_rsa_devops azureuser@<master-ip>

# Vrifier les permissions
chmod 600 terraform/id_rsa_devops
```

9.10.3 Pods non Ready

```
# Voir les vnements
kubectl describe pod <pod-name>

# Voir les logs
kubectl logs <pod-name>

# Recrer les pods
kubectl rollout restart deployment/task-manager-api
```

Chapitre 10

Maintenance, Monitoring et Meilleures Pratiques

10.1 Monitoring des applications

10.1.1 Endpoins de santé

L'application expose un endpoint health check :

```
curl http://master-ip:30080/health
# Rponse: {"status": "healthy"}
```

10.1.2 Kubernetes health checks

Readiness Probe

Configurée pour vérifier la disponibilité de l'application :

```
readinessProbe:
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 10
  periodSeconds: 5
```

Liveness Probe

À ajouter au deployment pour l'auto-healing :

```
livenessProbe:
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 30
  periodSeconds: 10
  failureThreshold: 3
```

10.2 Logging et debugging

10.2.1 Logs Kubernetes

```
# Logs actuels
kubectl logs <pod-name>

# Logs en temps rel
kubectl logs -f <pod-name>

# Logs pour tous les pods du dploiement
kubectl logs -l app=task-manager

# Logs du pod prcdent (si crash)
kubectl logs <pod-name> --previous
```

10.2.2 Inspection des ressources

```
# Dtails Pod
kubectl describe pod <pod-name>

# Dtails Deployment
kubectl describe deployment task-manager-api
```

```

# Dtails Service
kubectl describe service task-manager-service

# vnements du cluster
kubectl get events
```

10.3 Gestion des versions

10.3.1 Versioning d'images

Utiliser le semantic versioning (MAJOR.MINOR.PATCH) :

```
# Tags multiples
docker tag task-manager:vv1.0.0 seiflegnioui/task-manager:v1.0.0
docker tag task-manager:v1.0.0 seiflegnioui/task-manager:latest
docker push seiflegnioui/task-manager:v1.0.0
docker push seiflegnioui/task-manager:latest
```

10.3.2 Déploiement de nouvelles versions

```
# Mise  jour du dploiement
kubectl set image deployment/task-manager-api \
  task-manager-api=seiflegnioui/task-manager:v2.0.0

# Ou avec apply + YAML modifi
kubectl apply -f kubernetes/deployment.yaml
```

10.4 Meilleures pratiques

10.4.1 Infrastructure

1. Utiliser des tagsimages spécifiques (jamais "latest" en prod)
2. Implémenter des Network Policies pour la sécurité

3. Utiliser des Storage Classes pour la persistance
4. Configurer des Resource Quotas par namespace
5. Utiliser RBAC (Role-Based Access Control)

10.4.2 Application

1. Implémenter des health checks
2. Utiliser des probes de liveness et readiness
3. Gérer les signaux SIGTERM pour un arrêt gracieux
4. Implémenter le circuit breaker et retry logic
5. Utiliser des logs structurés (JSON)

10.4.3 DevOps

1. Versioner toute l'infrastructure (IaC)
2. Implémenter des tests en CI/CD
3. Utiliser des semantic versioning pour les releases
4. Sauvegarder les états Terraform
5. Documenter les runbooks

10.4.4 Sécurité

1. Scannner les images avec Trivy/Aqua
2. Utiliser des namespaces Kubernetes
3. Implémenter Network Policies
4. Chiffrer les secrets (Sealed Secrets/Vault)
5. Auditerance les accès avec RBAC

10.5 Cleanup et destruction

10.5.1 Destruction de l'infrastructure

```
# Depuis le rpertoire terraform
terraform destroy --auto-approve
```

Cette commande supprime :

— Les VMs Azure

— Le Virtual Network

— Le Network Security Group

— Les adresses IP publiques

— Tous les disques associés

Attention : Cette opération supprime définitivement tous les résources.

Chapitre 11

Conclusions et Perspectives

11.1 Résumé du projet

Ce projet a démontré avec succès l'implémentation d'une pile DevOps complète et moderne :

— Déploiement d'une application Flask REST

— Containerisation avec Docker

— Infrastructure as Code avec Terraform

— Configuration automation avec Ansible

— Orchestration Kubernetes (K3s)

— Pipeline CI/CD automatisé

— Documentation complète et guides

11.2 Technologies et outils maîtrisés

1. **Langage** : Python (Flask, Gunicorn)
2. **Containerisation** : Docker
3. **Cloud** : Microsoft Azure
4. **IaC** : Terraform
5. **Configuration** : Ansible
6. **Orchestration** : Kubernetes (K3s)
7. **CI/CD** : GitHub Actions

8. **Contrôle de version** : Git

11.3 Améliorations futures

11.3.1 Court terme

— Ajouter des tests unitaires et d'intégration
— Implémenter un monitoring avec Prometheus/Grafana
— Ajouter une base de données persistante
— Configurer TLS/SSL pour la sécurité
— Implémenter le logging avec ELK Stack

11.3.2 Moyen terme

— Migrer vers AKS (Azure Kubernetes Service)
— Implémenter Helm pour la gestion des charts
— Ajouter ArgoCD pour GitOps
— Configurer des Network Policies
— Implémenter Service Mesh (Istio/Linkerd)

11.3.3 Long terme

— Migration vers le multi-cloud
— Disaster recovery et backup automatisé
— Implémentation de chaos engineering
— ML Ops pour les modèles de ML
— Serverless pour les opérations ponctuelles

11.4 Leçons apprises

11.4.1 Points clés

1. L'automatisation est essentielle en DevOps
2. Infrastructure as Code rend l'infrastructure versionnable

3. La conteneurisation améliore la reproductibilité
4. L'orchestration Kubernetes est puissante mais complexe
5. Les pipelines CI/CD accélèrent très significativement les déploiements
6. La documentation est cruciale pour la maintenance

11.4.2 Défis rencontrés

— Configuration SSH avec Ansible sur Azure
— Gestion des IPs dynamiques et templates
— Synchronisation des tokens dans l'orchestration
— Configuration des probes de santé appropriées

11.5 Recommandations

Pour toute organisation déployant une architecture similaire :

1. Commencer simple et itérer progressivement
2. Automatiser depuis le début
3. Investir dans la documentation
4. Utiliser des standards de l'industrie
5. Implémenter la monitoring et les alertes
6. Faire des tests de disaster recovery régulièrement
7. Maintenir une culture DevOps forte

Annexe A

Code Source Complet

A.1 Application Flask (app.py)

```
from flask import Flask, jsonify, request, render_template
import uuid

app = Flask(__name__)

tasks = []

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks), 200

@app.route('/tasks', methods=['POST'])
def create_task():
    data = request.get_json()
    if not data or not 'title' in data:
        return jsonify({'error': 'Title is required'}), 400

```

```
    task = {
        'id': str(uuid.uuid4()),
        'title': data['title'],
        'description': data.get('description', ''),
        'done': False
    }
    tasks.append(task)
    return jsonify(task), 201

@app.route('/tasks/<task_id>', methods=['PATCH', 'PUT'])
def update_task(task_id):
    data = request.get_json()
    for task in tasks:
        if task['id'] == task_id:
            if 'done' in data:
                task['done'] = data['done']
            return jsonify(task), 200
    return jsonify({'error': 'Task not found'}), 404

@app.route('/tasks/<task_id>', methods=['DELETE'])
def delete_task(task_id):
    global tasks
    tasks = [task for task in tasks if task['id'] != task_id]
    return jsonify({'result': True}), 200

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
```

Annexe B

Configuration Terraform

B.1 Fichier main.tf complet

Voir le fichier `terraform/main.tf` dans le repository pour la configuration complète des ressources Azure.

Annexe C

Manifests Kubernetes

C.1 Déploiement complet

Voir le répertoire `kubernetes/` pour tous les manifests YAML.

Annexe D

Résumé des commandes

D.1 Commandes essentielles

|p4cm|p5cm|

Tâche	Commande

(Continued on next page)

Init Terraform	`terraform init`
Plan Terraform	`terraform plan`
Apply Terraform	`terraform apply`
Destroy infra	`terraform destroy`
Run Ansible	`ansible-playbook -i inventory.ini playbook.yml`
Build Docker	`docker build -t name:tag .`
Push Docker	`docker push name:tag`
Deploy K8s	`kubectl apply -f kubernetes/`
Get pods	`kubectl get pods`
Get logs	`kubectl logs pod-name`
Describe pod	`kubectl describe pod pod-name`
Port forward	`kubectl port-forward pod-name 5000:5000`
Scale deployment	`kubectl scale deployment name -replicas=3`
Restart deployment	`kubectl rollout restart deployment/name`

Annexe E

Ressources et références

E.1 Documentation officielle

- Terraform Documentation : https://www.terraform.io/docs
- Kubernetes Documentation : https://kubernetes.io/docs
- Ansible Documentation : https://docs.ansible.com
- Docker Documentation : https://docs.docker.com
- Azure Documentation : https://docs.microsoft.com/azure
- GitHub Actions : https://docs.github.com/en/actions

E.2 Livres recommandés

- "The DevOps Handbook" - Gene Kim, Patrick Debois
- "Infrastructure as Code" - Kief Morris
- "Kubernetes in Action" - Marko Luksa
- "The Phoenix Project" - Gene Kim, Kevin Behr

E.3 Community et forums

- Stack Overflow : https://stackoverflow.com/
- Reddit r/devops : https://reddit.com/r/devops/
- CNCF Slack : https://cloud-native.slack.com/

Conclusion Générale

Ce projet représente une démonstration complète des principes et outils DevOps modernes. À travers la conception, l'implémentation et le déploiement d'une architecture cloud entièrement automatisée, nous avons acquis une expérience pratique des technologies les plus exigeantes du secteur.

L'architecture proposée est scalable, sécurisée et durable, capable de supporter une croissance future et des modifications. Les pipelines CI/CD automatisés garantissent une livraison rapide et fiable des mises à jour logicielles.

Les apprentissages de ce projet s'étendent bien au-delà des outils spécifiques utilisés - ils englobent les principes fondamentaux du DevOps : l'automatisation, la collaboration, la mesure et le partage. Ces principes resteront pertinents peu importe l'évolution technologique.

Nous sommes convaincus que cette fondation solide permettra une croissance future et una adaptation aux besoins changeants de l'industrie.

Rapport généré le 25 mars 2026

Module MEIA - Cloud DevOps

Version 1.0